

MARCIAH: An Automated Q6 Workflow for Protein–Ligand Linear Interaction Energy Calculations

Marcia Yineth Castillo,[†] Gian Pietro Miscione,[†] Hugo Gutiérrez-de-Terán,[‡] and
Jordi Villà-Freixa^{*,¶,§}

[†]*Departamento de Química, Universidad de los Andes, Bogotá 111711, Colombia*

[‡]*Centro de Investigación en Nanomateriales y Nanotecnología (CINN), Consejo Superior de Investigaciones Científicas (CSIC), Edificio FINBA-ISPA, 33011 Oviedo, Asturias, Spain*

[¶]*Universitat de Vic – Universitat Central de Catalunya, 08500 Vic, Spain*

[§]*Institute for Research and Innovation in Life and Health Sciences in Central Catalonia (IRIS-CC), 08500 Vic, Spain*

E-mail: jordi.villa@uvic.cat

Abstract

Linear interaction energy (LIE) calculations provide a practical route to ligand binding-affinity estimation, but routine application still depends on consistent system setup, replicated molecular dynamics (MD), convergence assessment, and standardized postprocessing. MARCIAH (Modular Automated Route for Convergence-assessed Interaction Analysis and Handling) is a Q6-based workflow that automates these steps for protein–ligand LIE calculations. The implementation covers ligand and complex preparation, equilibration, replicated production MD, quality control of the interaction-energy averages entering the LIE model, optional extension of insufficiently converged

simulations, ligand-wise free-energy evaluation, and downstream regression analysis. MARCIAH combines shell and Python utilities for Q6 environment configuration, generation of Q6-ready inputs, parsing of electrostatic and van der Waals interaction energies from Q6 production logs, threshold-based stability screening, and export of final ΔG estimates in machine-readable form. The software is distributed with representative example files, a Conda environment for the analysis layer, and command-line entry points that separate MD generation from LIE analysis so that existing trajectories can be reused directly. The official ligand-parameterization route uses Maestro/Schrödinger together with QLigFEP, while the repository also includes an optional open-source pre-processing alternative based on Open Babel and ACPYPE for GAFF/Amber-style ligand and parameter generation. MARCIAH therefore provides a reproducible application layer around Q6 and QLigFEP for routine protein–ligand LIE studies in HPC environments.

1 Introduction

The linear interaction energy (LIE) method estimates ligand binding free energies from averaged electrostatic and van der Waals interaction terms obtained from end-state simulations.^{1,2} Its appeal in drug-discovery settings comes from a favorable balance between computational cost, physical transparency, and compatibility with ligand-series studies. In practice, however, a publishable LIE study depends on more than the final equation. Users must prepare ligand and complex systems consistently, generate input files for multiple simulation stages, manage replicated molecular dynamics runs, verify that the relevant interaction energies are stable enough for analysis, and only then compute and compare ΔG values.

Q6 provides the simulation engine needed for LIE and related free-energy calculations,³ while QLigFEP provides a route to ligand parameterization and related setup tasks. What is often missing is a coherent application layer that joins these capabilities into a workflow that can be followed, audited, and scaled across multiple ligands. Many groups still rely on

local notes, one-off scripts, and manually edited inputs, which makes reproducibility fragile and greatly increases the effort required to extend a study from a single ligand to a series.

MARCIAH was developed to address this practical gap. Rather than providing only isolated helper scripts, it organizes the full LIE procedure as a single executable application layer around Q6. Ligand parameterization, ligand and complex preparation, equilibration and production templates, interaction-energy analysis, extension of insufficiently converged simulations, LIE calculation, and regression analysis are handled within one workflow. The central contribution is therefore workflow integration: MARCIAH turns a scattered sequence of Q6-dependent tasks into a repeatable protocol for protein–ligand LIE calculations.

2 Workflow Overview and Implementation

MARCIAH implements a single end-to-end workflow organized around the two simulation contexts required by LIE: ligand-in-water simulations and protein–ligand complex simulations. Operationally, the software separates environment configuration, dataset staging, Q6 preparation and MD execution, and LIE analysis of existing production logs.

For the ligand-in-water setup, MARCIAH starts from ligand PDB structures and parameterizes them through a command-line Maestro/Schrödinger environment together with the `opls2Q.py` utility from QLigFEP. This official execution path generates ligand library files, ligand parameter files, and merged `OPLS2005_#_all.prm` files. MARCIAH then converts these inputs into Q6-ready files through `Qprep6` and generates the ligand FEP file from the prepared ligand-water structure. The repository also includes an optional open-source ligand-preparation route based on Open Babel and ACPYPE, which generates GAFF/Amber-style ligand files suitable for preprocessing and inspection but does not yet replace the current OPLS/Q6 input branch directly. The MD stage generates equilibration inputs, prepares production inputs, and runs replicated ligand simulations. The resulting production logs can be passed directly to the analysis stage without any additional file conversion.

For the protein–ligand complex setup, the same preparation layer combines the starting protein structure with the ligand-specific input files, builds the complex input structure, generates the Q6 preparation input, and produces the topology and coordinate files required for MD. The current implementation accepts either a shared protein structure or ligand-specific protein structures, which makes it possible to import benchmark datasets assembled from multiple crystallographic complexes. Equilibration and replicated production are then executed for the complex state in the same manner as for the ligand state.

Figure 1 summarizes the full application path. Environment initialization is handled through helper scripts that verify Q6 availability, register the Schrödinger command-line environment, configure the required shell variables, and create the Python analysis environment. Separate command-line entry points are provided for dataset acquisition and staging, ligand parameterization, the MD stage, and the LIE stage. This separation allows existing ligand parameters or MD trajectories to be reused directly without rerunning earlier stages.

Once ligand and complex MD logs are available, `scripts/run_lie_analysis.sh` calls `check_high_errors.py` and `analyze_LIE_noqgui.py` on the selected ligand and complex log directories. The analysis layer extracts the Q-protein and Q-water interaction energies, compares first-half and second-half averages to identify simulations whose estimated error exceeds 1 kcal/mol, and evaluates the LIE estimate from the stabilized interaction-energy averages. This criterion follows practical guidance used in LIE applications.²

If the threshold is not met, MARCIAH generates extended production inputs so that the simulation can be continued without rebuilding the full system. This feature turns convergence checking from a manual decision into an explicit scripted stage of the workflow.

Once the production logs satisfy the stability requirement, `analyze_LIE_noqgui.py` parses ligand and complex log files through the shared `mdlog_energies.py` module and computes the LIE estimate

$$\Delta G_{\text{bind}} = \beta \left[\left\langle V_{\text{el}}^{\text{complex,wat}} + V_{\text{el}}^{\text{complex,prot}} \right\rangle - \left\langle V_{\text{el}}^{\text{lig,wat}} \right\rangle \right] + \alpha \left[\left\langle V_{\text{vdW}}^{\text{complex,wat}} + V_{\text{vdW}}^{\text{complex,prot}} \right\rangle - \left\langle V_{\text{vdW}}^{\text{lig,wat}} \right\rangle \right] + \gamma \quad (1)$$

from the extracted Q-protein and Q-water interaction terms. Multi-ligand execution is supported through a batch launcher, while the bundled command-line test writes a CSV summary for the example system. In the current implementation, the empirical coefficients are exposed directly at execution time, with example values $\alpha = 0.68$, $\beta = 0.11$, and $\gamma = 0.0$, making calibration choices transparent to the user.

The software also includes downstream evaluation utilities for reference ligands, including linear regression between experimental and calculated free energies with reporting of R^2 , MAE, and RMSE. MARCIAH therefore extends beyond simulation setup and free-energy extraction by providing an explicit postprocessing path for reference-ligand validation.

A functional MARCIAH installation requires access to the Q6 executables used to run equilibration and production MD, a Conda-managed Python environment capable of executing the analysis scripts, and a shell environment that can run the supplied helper scripts. Slurm is supported by several launchers for routine multi-ligand HPC use, but the bundled CLI test is executed directly from the command line and does not depend on a scheduler.

From the standpoint of reproducible execution, the software distinguishes clearly between user-supplied inputs, generated intermediate files, and analysis outputs. The minimum user-supplied inputs for one LIE calculation are (i) a prepared protein structure for the complex state, (ii) one or more ligand PDB structures with atom naming consistent across preparation and simulation stages, (iii) access to the Maestro/Schrödinger command-line environment together with `opls2Q.py` from QLigFEP for the official OPLS/Q6 ligand-parameterization route, (iv) topology and FEP files for the ligand and complex systems, and (v) production and equilibration templates that specify the Q6 simulation protocol. The repository also includes

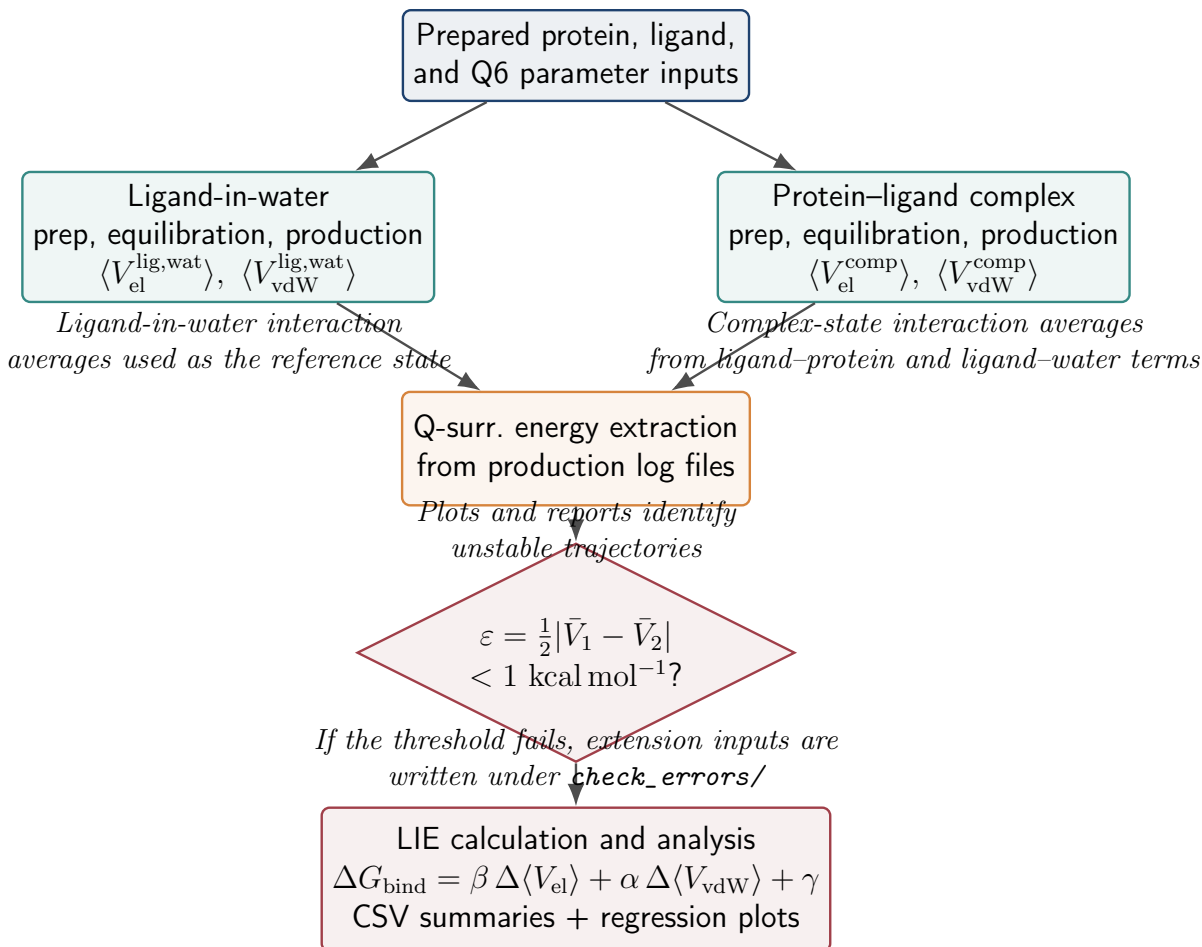


Figure 1: General scheme of the MARCIAH workflow. The workflow prepares ligand and complex systems separately, runs replicated Q6 equilibration and production simulations, checks the stability of the interaction-energy averages used in the LIE model, extends the MD simulations when the error threshold is not met, and finally computes LIE binding free energies and downstream analyses.

an optional Open Babel/ACPYPE preprocessing route for open-source ligand preparation, but this branch does not yet replace the official OPLS/Q6 path. The repository already contains representative examples of these file classes, which define the practical input contract for running the workflow.

The root distribution contains `environment.yml` together with a setup script that creates the Conda environment `marciah-q6-lie`, as well as an auxiliary Conda specification for the optional Open Babel/ACPYPE preprocessing route. A minimal setup sequence consists of cloning MARCIAH, registering the Maestro/Schrödinger command-line environment, config-

uring the analysis environment, ensuring that the Q6 binaries are available in the execution environment, staging either user-supplied structures or a supported benchmark dataset into the MARCIAH directory layout, running ligand parameterization on the staged ligand PDB files, and then running either the bundled command-line test or the full ligand and complex preparation, equilibration, production, quality-control, and LIE-analysis steps.

Each workflow stage is defined by a compact set of required inputs, an executable script or command, and a corresponding set of outputs. This input/output structure makes the execution model explicit: ligand and complex preparation produce Q6-ready topology, coordinate, and FEP files; equilibration produces restart files for the production stage; production generates MD logs and energy files; quality-control scripts report interaction-energy stability; and the final analysis writes ligand-wise LIE estimates to CSV.

3 Usability Advances

The principal advance of this application is that it formalizes the entire LIE protocol, not only the final energy calculation. The repository provides explicit module boundaries for ligand preparation, complex preparation, equilibration, production, quality control, free-energy estimation, and regression analysis. This reduces hidden know-how and makes the protocol easier to hand off between users.

The second major advance is the integration of quality control into the default workflow. Many practical LIE pipelines compute ΔG directly from whatever production trajectory is available. Here, the scripts explicitly inspect the electrostatic and van der Waals interaction-energy averages entering the LIE estimate, generate per-log plots, identify simulations that exceed an error threshold, and support extension of the simulation length before free-energy reporting. This is a scientifically important usability feature because it encourages users to reject unstable trajectories rather than accept them silently.

Third, the workflow is cluster-oriented and scriptable. Slurm launchers are supplied for

preparation, production, extension, and LIE evaluation, which makes the application suitable for routine academic HPC use. The output is machine-readable, primarily as CSV tables and report files, so the calculated energies can be integrated directly into later benchmarking or medicinal-chemistry analysis steps.

Finally, the software exposes empirical choices instead of burying them. Simulation lengths, coefficient values, and directory expectations are visible in the scripts and repository structure, which makes the protocol easier to audit and adapt.

4 Demonstration Case in the Current Repository

The executable distribution contains two complementary examples. The first is a worked example spanning ligand and complex setup files, production logs, quality-control scripts, and command-line LIE result generation. The second is a structure-derived benchmark path that downloads BACE1 inhibitor complexes linked to BindingDB and stages them into the MARCIAH directory layout. The repository also contains a reference-ligand regression utility for comparing calculated and experimental free energies. Together, these examples cover both immediate installation testing and benchmark construction from external structural data.

The bundled log-only demonstration follows an end-to-end MDM2 inhibitor workflow: prepared protein and ligand structures are converted into ligand and complex Q6 systems, the systems are equilibrated and propagated through replicated production simulations, the stability of the interaction-energy averages entering the LIE model is evaluated, insufficiently stable simulations can be extended, ligand-wise LIE ΔG values are computed, and selected ligands can be compared against experiment through the regression utilities. In addition, the repository now provides a BindingDB-derived BACE1 benchmark path that downloads selected crystallographic complexes, extracts ligand and protein structures, infers preparation anchors, and then reuses the same MARCIAH launchers for parameterization, MD, and LIE analysis. BACE1 was chosen because it is a historically important LIE test system.^{4,5}

For first-time users, the shortest validation path remains `scripts/setup_conda_env.sh` together with `scripts/run_cli_example_test.sh`, which execute the analysis layer on bundled example logs.

5 Data and Software Availability

MARCIAH is distributed through a public GitHub repository. Repository URL: <https://github.com/JordiVillaFreixa/MARCIAH>. The distribution contains the analysis and setup scripts, representative ligand and complex example files, dataset manifests and acquisition helpers for a BindingDB-linked BACE1 benchmark, the Conda environment specification in `environment.yml`, an auxiliary Conda environment for the Open Babel/ACPYPE route, installation and command-line test helpers in `scripts/`, a repository-level BibTeX citation record in `CITATION.bib`, and the root-level `README.md` that defines the required directory layout, software metadata, and execution sequence. The software is distributed under the MIT license.

The current implementation assumes an HPC-style environment with Python 3.11 in the supplied Conda environment and Slurm available for execution of the supplied batch scripts. The full workflow targets Linux-based cluster execution, whereas the bundled command-line analysis example provides a lightweight installation and validation path for the analysis layer. Example ligand and complex files, together with production logs for a minimal test case, are included directly in the repository. Q6 remains an external dependency for equilibration and production MD; MARCIAH provides environment checking and installation helpers, but successful Q6 compilation still depends on the local platform and compiler toolchain. In the official Q6 repository, macOS compilation is documented through the `osx` makefile target, but the documented path refers to older Mac OS X environments and local tests with current `gfortran` toolchains fail in the signal-handler code of `Qdyn6` and `Qpi6`. The official MARCIAH ligand-parameterization path depends on the command-line Maestro/Schrödinger en-

vironment together with QLigFEP, whereas the bundled open-source alternative based on Open Babel and ACPYPE currently provides GAFF/Amber-style ligand outputs that still require conversion before entering the OPLS/Q6 branch. These dependencies are cited and linked separately.^{3,6,7}

6 Associated Content

Supporting Information. Supporting Information includes the minimal command-line example distributed with MARCIAH, the directory schema expected by the analysis scripts, and a compact table mapping workflow stages to their required inputs and generated outputs.

7 Author Information

Corresponding Author

Jordi Villà-Freixa, Universitat de Vic – Universitat Central de Catalunya, Vic, Barcelona 08500, Spain; Email: jordi.villa@uvic.cat

Notes

The authors declare no competing financial interest.

8 Acknowledgments

The authors acknowledge the Q6 and QLigFEP software ecosystems, which provide the simulation and setup capabilities on which MARCIAH is built.

References

- (1) Aqvist, J.; Marelus, J. The Linear Interaction Energy Method for Predicting Ligand Binding Free Energies. *Comb. Chem. High Throughput Screen.* **2001**, *4*, 613–626.

- (2) Gutiérrez-de Terán, H.; Åqvist, J. *Computational Drug Discovery and Design*; Humana Press, 2012; pp 305–323.
- (3) Bauer, P.; Barrozo, A.; Purg, M.; Amrein, B. A.; Esguerra, M.; Wilson, P. B.; Major, D. T.; Åqvist, J.; Kamerlin, S. C. L. Q6: A Comprehensive Toolkit for Empirical Valence Bond and Related Free Energy Calculations. *SoftwareX* **2018**, 7, 388–395.
- (4) Tounge, B. A.; Reynolds, C. H. Calculation of the Binding Affinity of β -Secretase Inhibitors Using the Linear Interaction Energy Method. *J. Med. Chem.* **2003**, 46, 2074–2082.
- (5) Tounge, B. A.; Rajamani, R.; Baxter, E. W.; Reitz, A. B.; Reynolds, C. H. Linear Interaction Energy Models for β -Secretase (BACE) Inhibitors: Role of van der Waals, Electrostatic, and Continuum-Solvation Terms. *J. Mol. Graph. Model.* **2006**, 24, 307–319.
- (6) Q Users Q6 Repository – EVB, FEP and LIE Simulator. <https://github.com/qusers/Q6>, 2026; Accessed 2026-03-30.
- (7) Q Users QLigFEP Repository. <https://github.com/qusers/qligfep>, 2026; Accessed 2026-03-30.

TOC Graphic

